

CS516: Parallelization of Programs

Optimizing Data Transfers and Unified Memory

Vishwesh Jatala

Assistant Professor

Department of CSE

Indian Institute of Technology Bhilai

vishwesh@iitbhilai.ac.in



Outline

- Optimizing Memory Transfers
- Unified Memory

Data Transfer Using Streams

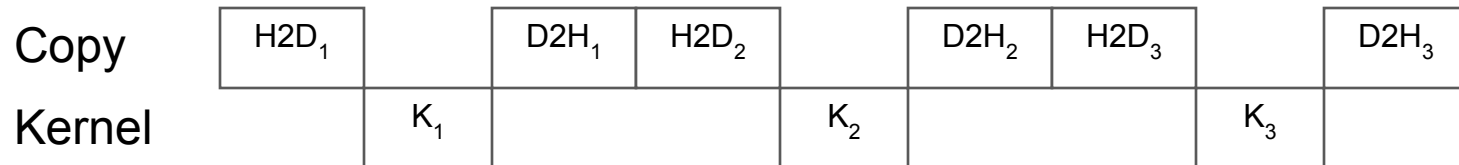
- Stream is sequence of operations that execute on the GPU device in the order in which they are issued by host
- Operations across different streams can be interleaved and can be executed concurrently.

Overlapping Data Transfer and Execution in Streams

```
for (int i = 0; i < nStreams; ++i) {  
    int offset = i * streamSize;  
    cudaMemcpyAsync (&d_a[offset], &a[offset], Bytes, cudaMemcpyHostToDevice,  
        stream[i]);  
    kernel<<<streamSize/blockSize, blockSize, 0, stream[i]>>>(d_a, offset);  
    cudaMemcpyAsync (&a[offset], &d_a[offset], Bytes, cudaMemcpyDeviceToHost,  
        stream[i]);  
}
```

Stream ₀	H2D ₁	K ₁	D2H ₁
Stream ₁	H2D ₂	K ₂	D2H ₂
Stream ₂	H2D ₃	K ₃	D2H ₃

Copy	H2D ₁	D2H ₁	H2D ₂	D2H ₂	H2D ₃	D2H ₃
Kernel	K ₁	K ₂	K ₃			



Overlapping Data Transfer and Execution in Streams

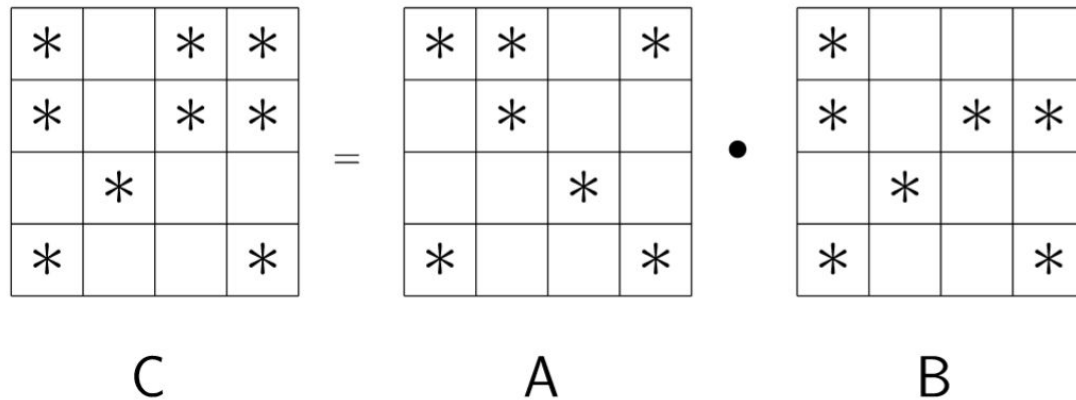
```
for (int i = 0; i < nStreams; ++i) {
    int offset = i * streamSize;
    cudaMemcpyAsync (&d_a[offset], &a[offset], Bytes, cudaMemcpyHostToDevice, stream[i]);
}
for (int i = 0; i < nStreams; ++i) {
    int offset = i * streamSize;
    kernel<<<streamSize/blockSize, blockSize, 0, stream[i]>>>(d_a, offset);
}
for (int i = 0; i < nStreams; ++i) {
    int offset = i * streamSize;
    cudaMemcpyAsync (&a[offset], &d_a[offset], Bytes, cudaMemcpyDeviceToHost, Stream[i]);
}
```

Stream ₀	H2D ₁	K ₁	D2H ₁
Stream ₁	H2D ₂	K ₂	D2H ₂
Stream ₂	H2D ₃	K ₃	D2H ₃

Copy	H2D ₁	H2D ₂	H2D ₂₃	D2H ₁	D2H ₂	D2H ₃
Kernel	K ₁	K ₂	K ₃			

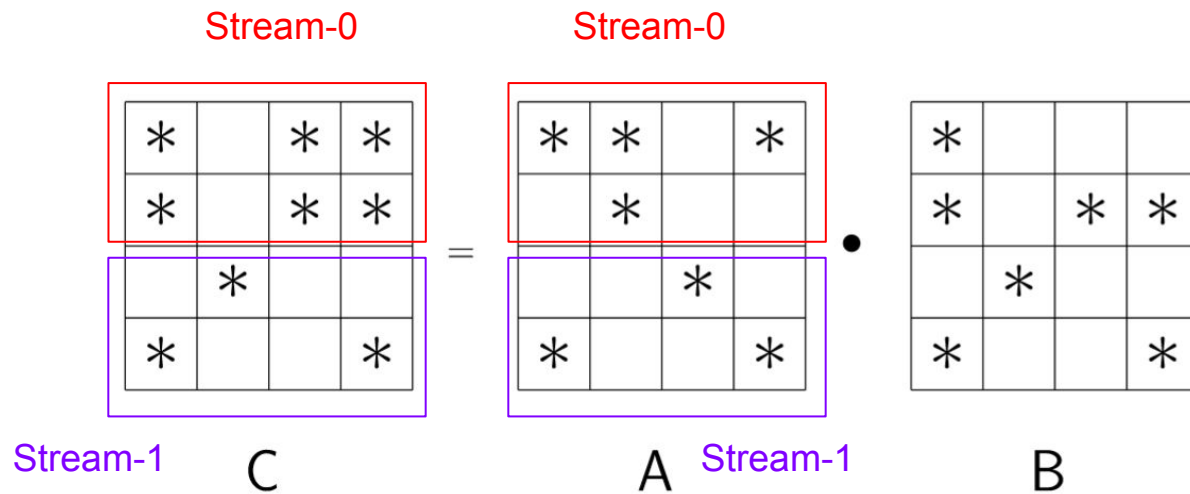
Copy	H2D ₁	H2D ₂	H2D ₃	D2H ₁	D2H ₂	D2H ₃
Kernel		K ₁	K ₂	K ₃		

Matrix Multiplication with Streams



$$C_{ij} = \sum_k A_{ik} \cdot B_{kj}$$

Matrix Multiplication with Streams



Outline

- Unified Memory

Memory Management - Unified Memory

- Unified memory is a single virtual address space that can be accessed by both CPU and GPU.
- Data transfers are automatic.

```
int main() {  
    h_a = malloc(..) // host array  
    cudaMalloc(d_a,...) // device  
  
    /* Initialize h_a */  
  
    cudaMemcpy(d_a, h_a, HostToDevice)  
  
    Kernel<<<1, N> (d_a);  
  
    cudaMemcpy(h_a, d_a, DevicetoHost)  
    process(h_a);  
  
    cudaFree(d_a);  
    free(h_a);  
}
```

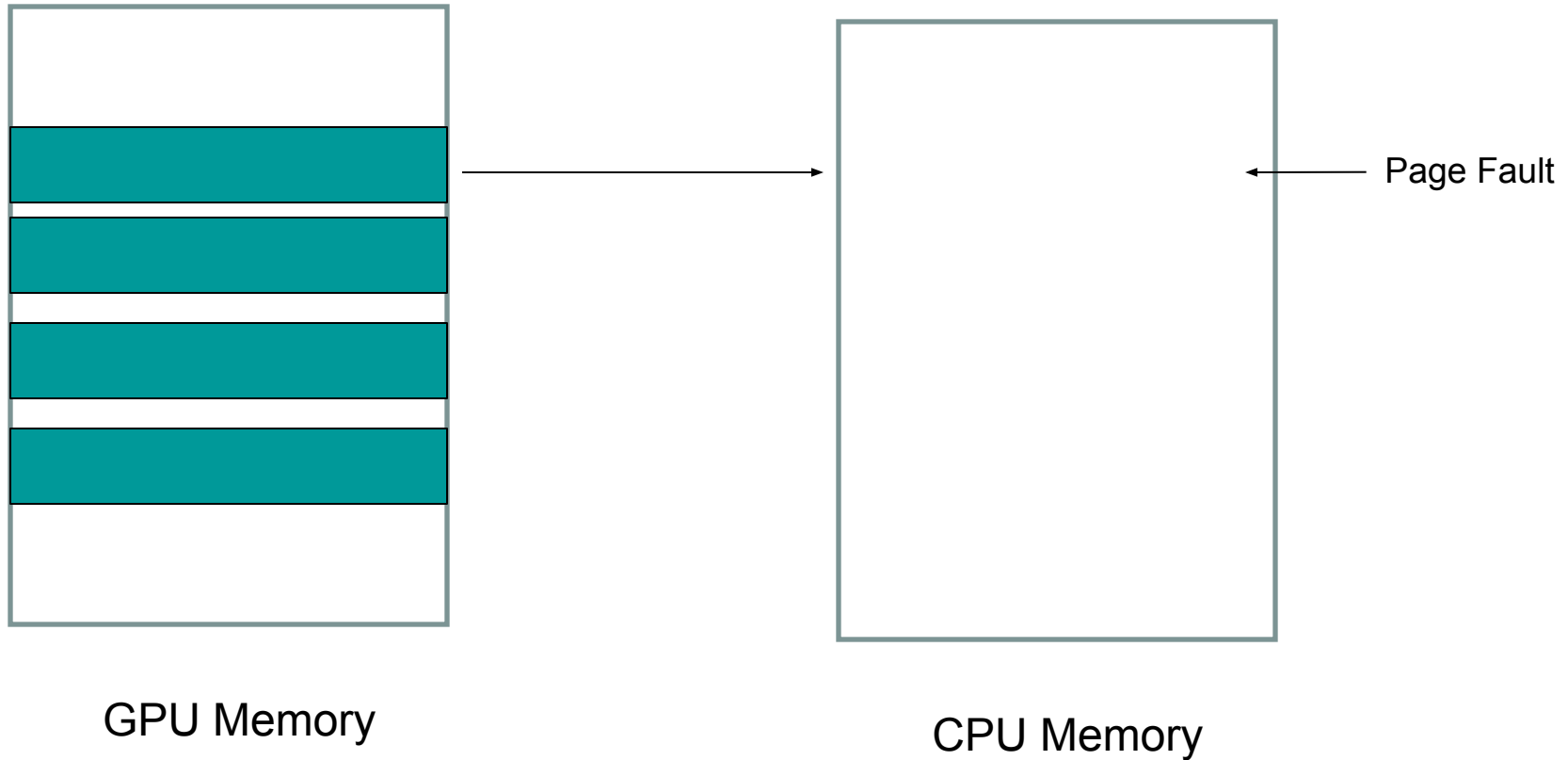
Without Unified Memory

```
int main() {  
  
    cudaMallocManaged(a,...)  
  
    /* Initialize a */  
  
    Kernel<<<1, N> (a);  
    cudaDeviceSynchronize();  
  
    process(a);  
  
    cudaFree(a);  
}
```

With Unified Memory

Unified Memory in Kepler

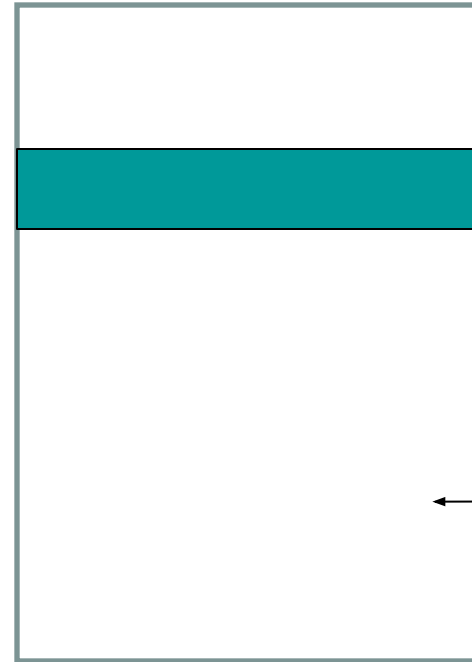
`cudaMallocManaged(a,...)`



Unified Memory in Kepler



GPU Memory

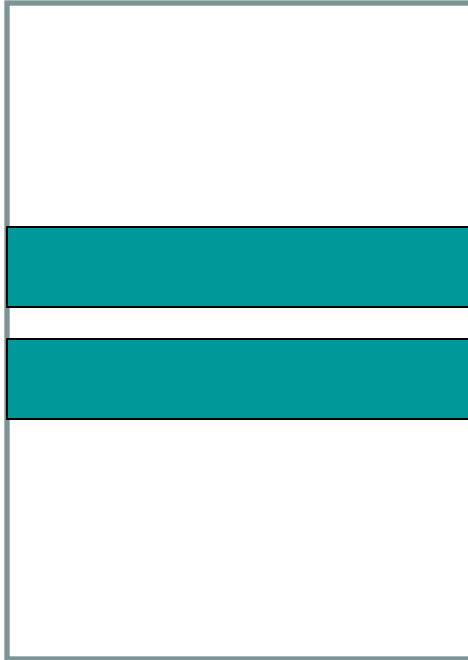


← Page Fault

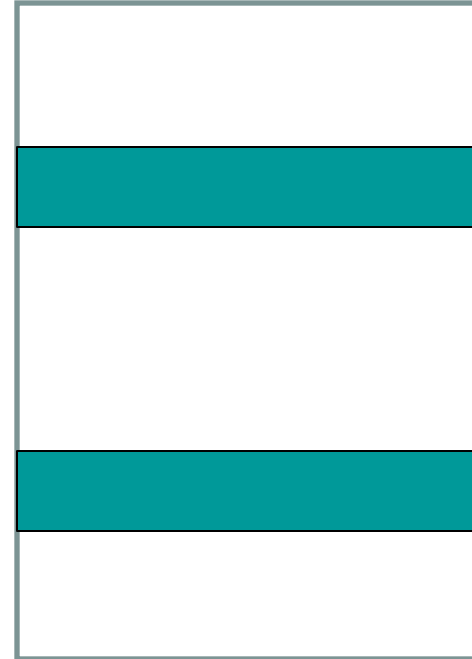
CPU Memory

Unified Memory in Kepler

Kernel Launch



GPU Memory



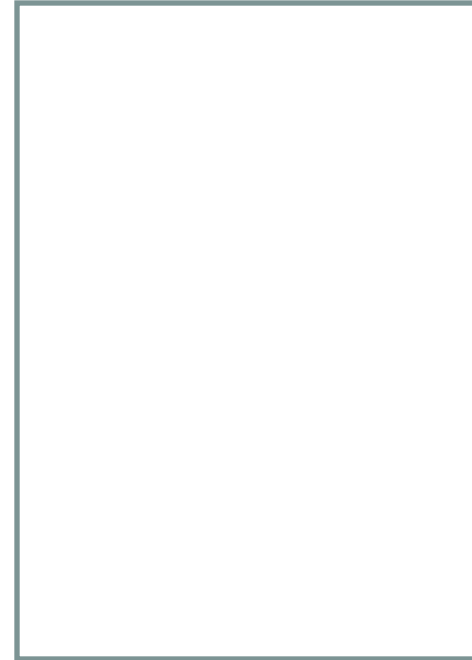
CPU Memory

Unified Memory in Kepler

Kernel Launch



GPU Memory



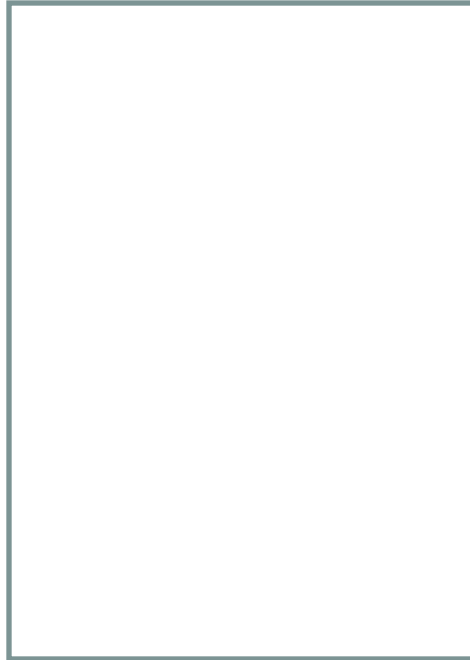
CPU Memory

Unified Memory in Kepler

- Memory is allocated on GPU upon `cudaMallocManaged()`
- All the pages must reside on the GPU before the kernel launch
- Page faults can occur in CPU but not in GPU
- No oversubscription of pages

Unified Memory in Pascal

Initially may not have any pages!



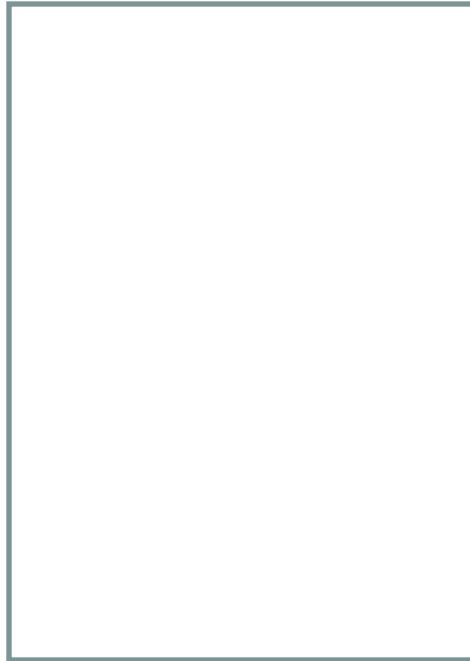
GPU Memory



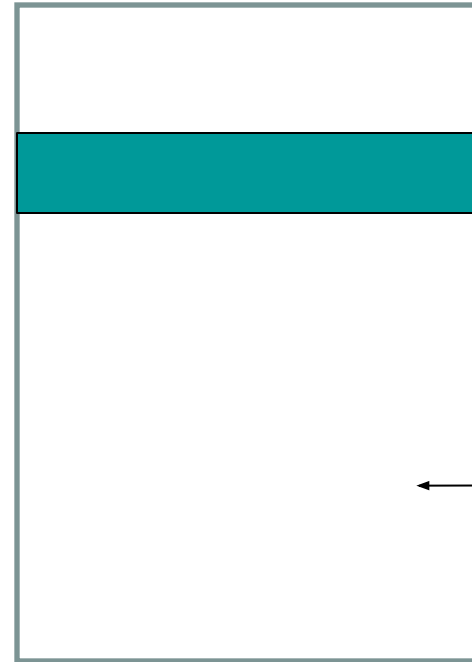
← Page Fault

CPU Memory

Unified Memory in Pascal



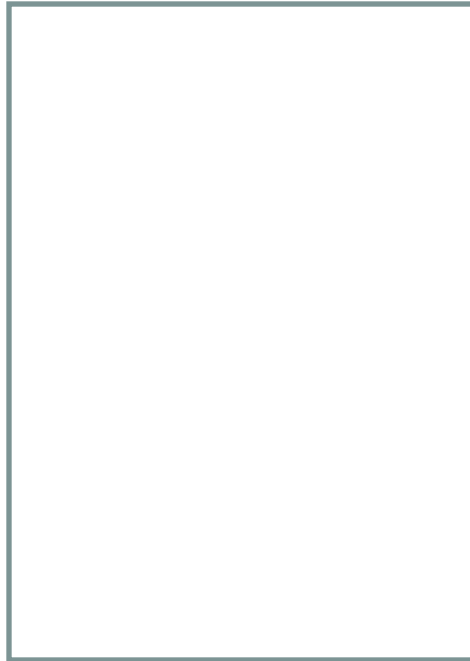
GPU Memory



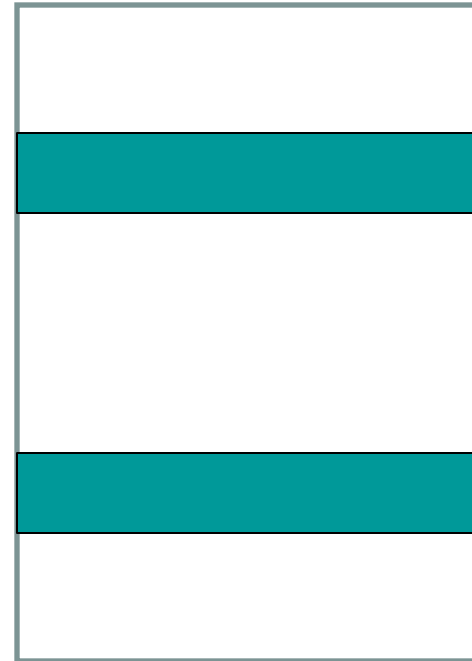
← Page Fault

CPU Memory

Unified Memory in Pascal

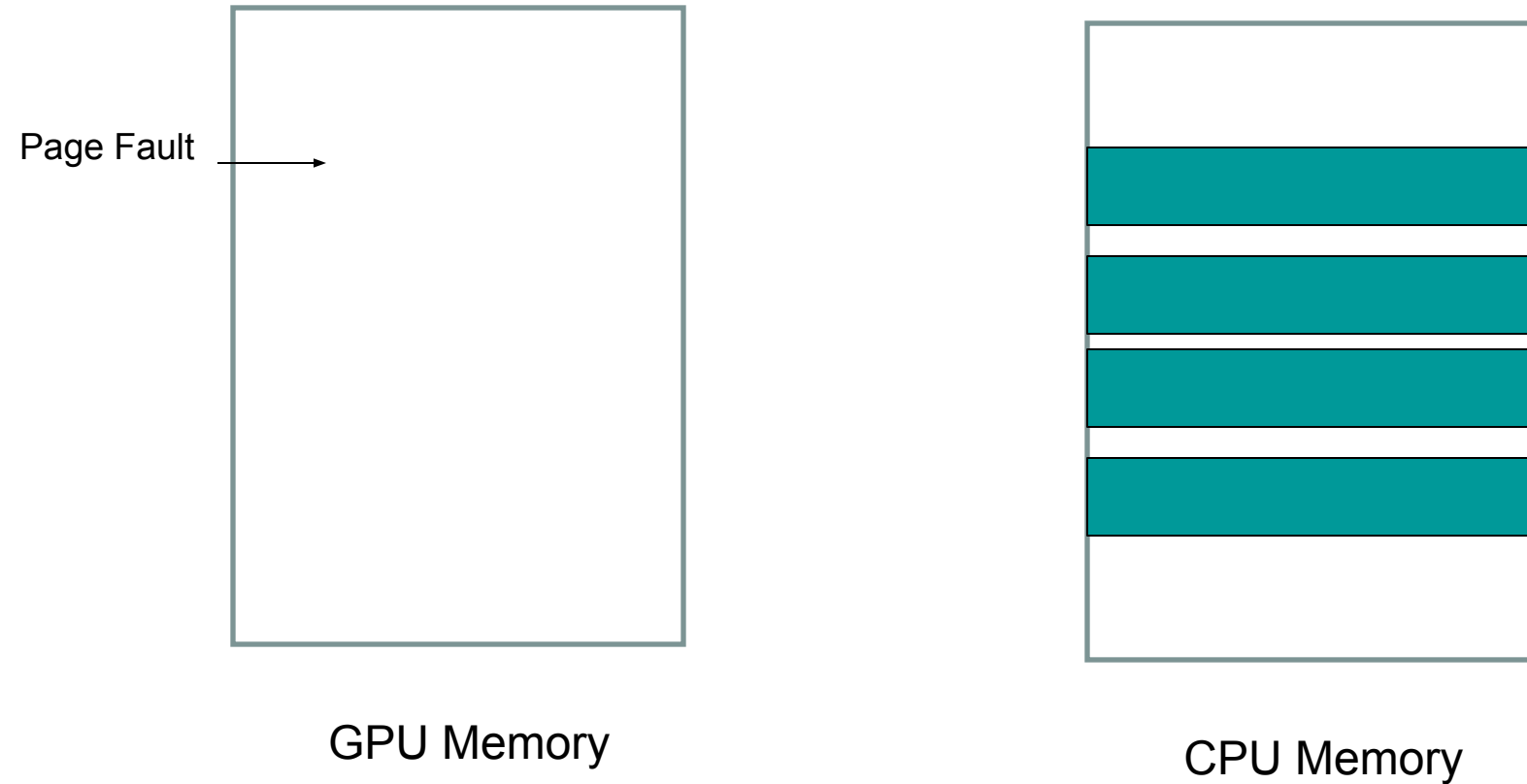


GPU Memory



CPU Memory

Unified Memory in Pascal



Unified Memory in Pascal



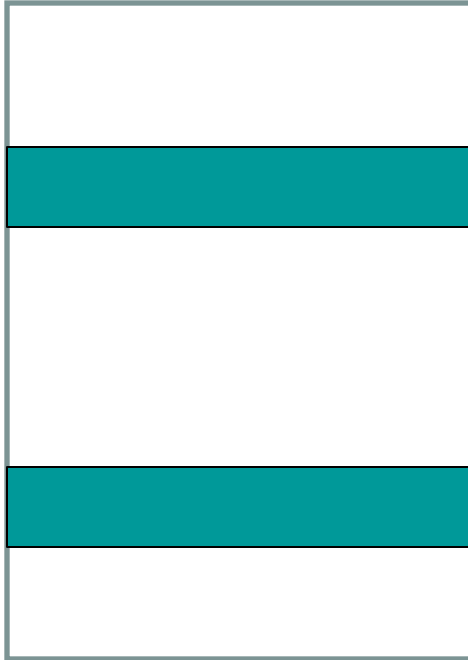
GPU Memory



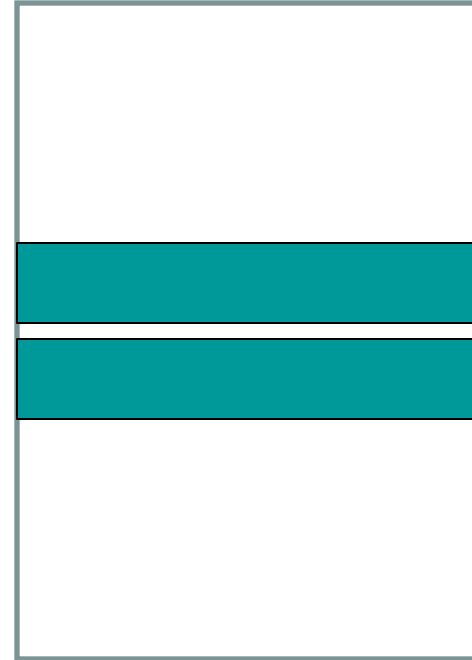
CPU Memory

Page Fault →

Unified Memory in Pascal



GPU Memory



CPU Memory

Unified Memory in Pascal

- Pages may not be allocated on GPU upon `cudaMallocManaged()`
- Page faults can occur both in CPU and GPU.
- Oversubscription of pages is possible by evicting the pages in GPU

Summary

- Optimizing Memory Transfers
 - Pinned Memory
 - Overlapping data transfers with streams